

项目报告

本项目是一个以信科的吉祥物“攻城狮”为主角的一个桌面宠物程序

程序功能介绍

- 实现了“攻城狮”的各种不同的动作形态
- 实现了ai的API的配置
- 实现了文件传输系统

各模块和类的设计细节

模块一：桌面宠物整体框架

类设计

类名	继承	职责
PetWindow	QWidget	主窗口：无边框透明置顶窗口，统合所有子模块，处理鼠标拖拽、右键菜单、文件拖放
PetController	QObject	有限状态机：驱动帧动画、随机动作调度（5s间隔）、强制状态锁定、拖拽态保存/恢复
AnimationManager	(无)	帧数据管理：从 资源目录加载各状态的 PNG 帧序列，提供 frame/interval 查询
TrayMenuManager	QObject	系统托盘：托盘图标 + 右键菜单（Show/Hide、AI开关、状态控制、退出），通过信号与 PetWindow 通信

核心技术点

- **窗口:** `FramelessWindowHint | Tool | WindowStaysOnTopHint + WA_TranslucentBackground`，通过 `QPainter::drawPixmap` 逐帧渲染。
- **状态机:** 7 种状态（IDLE/Dragged/Walking/Sleeping/Working/Celebrating/Sad），两个 `QTimer` 分别驱动帧更新（频率按状态可变）和随机动作（固定 5s）。
- **拖拽:** `mousePressEvent` 记录全局坐标偏移，移动超过系统阈值后进入 Dragged 态；释放时恢复原态。
- **随机行走:** 在屏幕可用区域内随机选目标点，沿直线以 `4px/frame` 速度插值移动。
- **睡眠循环:** 默认 3 次动画循环后自动回到 IDLE。

- **强制状态:** `setForcedState()` 锁定状态，阻止随机动作调度，通过 `clearForcedState()` 解锁。

模块二：AI API 调用

技术栈: Qt Network (`QNetworkAccessManager`) + OpenAI 兼容 API (默认 DeepSeek)

类设计

类名	继承	职责
AIChatManager	QObject	核心：API 配置管理、对话历史维护、HTTP 请求/响应处理、QSettings 持久化
AISettingsDialog	QDialog	设置 UI：enabled / API Key / endpoint / model / systemPrompt / temperature 表单编辑
ChatBubbleDialog	QDialog	气泡 UI：Input 模式（用户输入文本）和 Display 模式（展示 AI 回复，6.5s 自动关闭）

核心技术点

- **协议:** POST `{endpointUrl}`，Header 为 `Authorization: Bearer {apiKey}`，body 为 OpenAI Chat Completions 格式 JSON。
- **对话历史:** `QList<ChatMessage{role, content}>`，每次请求前 `buildMessagesPayload()` 序列化；`historyLimit` 限制上下文长度（默认 10 条），`trimHistory()` 从头裁剪。
- **API Key 存储:** Base64 编码 + `b64`：前缀后存入 `QSettings`（注册表），读取时解码还原（混淆级保护）。
- **错误处理:** 按 HTTP 状态码（401/429/5xx）分类给出中文错误提示，尝试解析响应体 `error.message`。
- **异步回调:** `QNetworkReply::finished` → lambda 解析 JSON → 提取 `choices[0].message.content` → emit `replyReady()`。
- **气泡定位:** 无边框异形对话框，背景用 `thoughtbubble.png` / `bubble.png`，定位在宠物右上角并自动修正屏幕边界。

模块三：文件传输功能

技术栈: `cpp-httplib 0.48.0`（HTTP 服务器）+ `libqrencode 4.1.1`（二维码）+ Qt 资源内嵌 HTML 前端

类设计

类名	继承	职责
FileTransferServer	QObject	HTTP 服务器：独立 QThread 中运行 cpp-httplib Server，提供 / 页面、/list API、/download 流式接口
FileTransferDialog	QDialog	传输窗口：展示二维码、IPv6 URL、文件状态、停止按钮
QRCodeGenerator	(静态类)	二维码：封装 libqrencode，generateQRCode(text, size) → QPixmap

核心技术点

- **线程模型:** moveToThread(&m_workerThread) 将服务器移至工作线程；QAtomicInt 跨线程查询状态；m_svr->stop() + quit() + wait(3000) 安全停止。
- **IPv6 获取:** 遍历 QNetworkInterface::allInterfaces()，过滤虚拟接口 (virtual/vmnet/docker 等) 和链路本地/组播/ULA 地址，返回全球公网 IPv6。
- **安全机制:**
 - UUID token 认证 (所有请求须带 ?token=xxx)。
 - resolveFilePath() 仅根据文件名在 m_sharedFiles 列表中匹配完整路径，防止路径遍历。
 - QMutex m_filesMutex 保护文件列表并发访问。
- **API 路由:**
 - GET / → 返回 file_browser.html (替换 {{TOKEN}} 模板占位符)。
 - GET /list → 返回 JSON [{name, size, mtime}]。
 - GET /download?file=xxx → 流式传输，64KB 缓冲分块，Content-Disposition: attachment。
- **动态添加:** 服务器运行期间 addFiles() 线程安全追加文件，手机刷新页面即可。
- **二维码:** Qrcode_encodeString8bit() → 逐模块 QPainter 绘制 → QPixmap，纠错级别 M。

项目分工

- 李子政主要完成了这个项目整体的大的框架，完成了通过图片来表示不同动作的功能，随着时间定时在不同的动作之间进行切换的功能以及利用ai完成了一些美术素材的创作
- 段治瀛完成了部分美术素材的创作，完成了关于ai的API的配置，完成了关于文件传输系统的制作

项目总结和反思

- 在设计项目的时候应该考虑好之后怎么添加新的功能，并且将一些公共的东西作为一个接口写好来方便两个人使用
- 在进行组队的这种作业的时候最好还是有一個人能够独立完成整个项目的大体框架，然后再去分配任务进行各种新功能的扩展
- 在借助ai的帮助写代码之后应该自己认真审查一遍。如果自己不了解的领域在使用的时候要格外小心，有的时候直接重新开始比起一直在那里改要好得多，当然当debug非常困难的时候这个东西真好用啊，代码变得更规范了
- 在做大作业的时候最好一次性就将要做完的东西直接做完，不然应该打好注释，以后应该改掉几乎不写注释的问题